

PostgreSQL para Analistas de Dados (Material de Estudos)

Parte 1 – Estrutura, Integridade e Manipulação de Dados

1. Por que Analistas de Dados precisam entender PostgreSQL?

Em empresas, dados geralmente estão espalhados por sistemas como:

- ERP (estoque, compras, faturamento)
- CRM (clientes, leads, vendas)
- Sistemas de logística (entregas, rastreios)
- Sistemas internos (cadastro de usuários, acessos, atendimento)

O PostgreSQL é usado para **armazenar e organizar** esses dados de forma estruturada, permitindo:

O que um analista faz com SQL no dia a dia:

- Verificar vendas do mês, produtos mais vendidos, clientes recorrentes
- Encontrar inconsistências no cadastro (e-mails vazios, valores negativos, datas erradas)
- Criar relatórios para BI e dashboards
- Extrair dados para análises em Excel, Power BI ou Python

Então SQL não é “só programação” — é uma forma de conversar com a base de dados e responder perguntas do negócio.

2. Tipos de Dados mais comuns e importantes no PostgreSQL

Os **tipos de dados** definem o **formato e as regras básicas** do que uma coluna pode armazenar.

Se você escolhe um tipo errado, pode ter problemas como:

- não conseguir filtrar corretamente
 - erros de inserção
 - cálculo errado no relatório
 - dados inconsistentes no BI
-

2.1 Tipos numéricos (cálculo, métricas e valores)

INTEGER

Usado para quantidades e valores inteiros:

- quantidade de produtos
- idade
- parcelas

`quantidade` **INTEGER**

 Situação real:

Uma coluna `quantidade` precisa ser inteira, porque faz sentido vender **3 unidades**, não **3.7 unidades**.

NUMERIC(p,s)

Usado quando você precisa de **precisão decimal**, principalmente dinheiro.

```
preco NUMERIC(10,2)
```

 Situação real:

Para análises financeiras, usar **NUMERIC** é mais seguro do que tipos aproximados, pois evita arredondamentos indesejados em somatórios.

SERIAL

O **SERIAL** é usado para gerar **códigos automáticos**, muito comum em IDs.

```
id_produto SERIAL
```

 Situação real:

Quando o sistema cadastra milhares de pedidos, não dá para alguém ficar digitando o ID manualmente.

2.2 Tipos de texto (cadastros e descrições)

VARCHAR(n)

Texto com tamanho limite, ideal para campos curtos como:

- nome
- email
- telefone

```
nome VARCHAR(100)
```

```
email VARCHAR(150)
```

 Situação real:

Limitar o tamanho evita “poluição de dados”, como alguém digitando uma descrição enorme no campo de nome.

TEXT

Texto livre, sem limite definido (na prática é bem grande).

Bom para:

- observações
- descrição de produto
- comentários

`observacao TEXT`



Situação real:

Em uma tabela de atendimento ao cliente, você pode precisar salvar mensagens longas no campo `observacao`.

2.3 Tipos de data e hora (tempo é tudo em dados)



DATE

Armazena apenas a data:

`data_pedido DATE`



Situação real:

Relatório de “vendas por dia” usa DATE.



TIMESTAMP

Armazena data + hora:

`criado_em TIMESTAMP`

📌 Situação real:

Quando a empresa quer saber:

- “qual horário tem mais vendas?”
 - “quanto tempo demora entre pedido e entrega?”
-

2.4 BOOLEAN (sim/não)

Usado para estados simples:

- ativo/inativo
- pago/não pago

ativo BOOLEAN

📌 Situação real:

Filtrar clientes ativos para campanhas de marketing.

✅ 3. Regras de Integridade: “como garantir que os dados não virem bagunça”

As regras de integridade servem para evitar erros do tipo:

- pedido sem cliente
- preço negativo
- produto sem nome
- data de nascimento no futuro

Em bancos reais, isso é MUITO comum — então regras protegem a qualidade do dado na origem.

3.1 PRIMARY KEY (identidade única)

A chave primária identifica cada linha de forma única.

```
id_cliente SERIAL PRIMARY KEY
```

📌 Situação real:

Se existem dois clientes chamados “João”, o sistema ainda consegue diferenciar quem é quem pelo ID.

3.2 NOT NULL (campo obrigatório)

Evita cadastros incompletos.

```
nome VARCHAR(100) NOT NULL
```

📌 Situação real:

Cadastrar cliente sem nome atrapalha:

- relatórios
 - buscas
 - emissão de nota
 - integração com sistemas
-

3.3 UNIQUE (valor não pode repetir)

Garante que certo valor seja exclusivo.

```
email VARCHAR(150) UNIQUE
```

📌 Situação real:

Evita duplicidade de cadastro (mesmo e-mail em vários clientes).

3.4 CHECK (regras de validação)

Garante condições lógicas.

CHECK (preço > 0)

📌 Situação real:

Evita produto com preço zero ou negativo, que destrói relatórios de faturamento.

✅ 4. Comandos DDL (estrutura do banco)

DDL é usado quando você vai:

- criar tabelas novas
- corrigir estrutura
- adaptar o banco para novas necessidades

📌 Exemplo de cenário:

A empresa começou vendendo apenas produtos físicos, mas agora vende **serviços também**. O banco precisa evoluir.

4.1 CREATE TABLE (criar uma tabela)

Exemplo: sistema simples de vendas

```
CREATE TABLE cliente (  
    id_cliente SERIAL PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE,  
    data_cadastro DATE NOT NULL  
);
```

 Situação real:

Tabela necessária para relatórios como:

- “quantos clientes novos por mês?”
 - “qual percentual possui e-mail cadastrado?”
-

4.2 ALTER TABLE (alterar estrutura)

Adicionar coluna: telefone

```
ALTER TABLE cliente  
ADD COLUMN telefone VARCHAR(20);
```

 Situação real:

O time comercial pediu o telefone para disparo de WhatsApp.

Alterar tipo de coluna

```
ALTER TABLE cliente  
ALTER COLUMN nome TYPE VARCHAR(150);
```

 Situação real:

Descobriram que 100 caracteres é pouco para alguns nomes longos e corporativos.

4.3 DROP TABLE (apagar tabela)

```
DROP TABLE cliente;
```


⚠ Usado com extrema cautela.

📌 Situação real:

Geralmente usado apenas em ambientes de teste ou quando um projeto foi abandonado.

✅ 5. Foreign Key: como garantir relacionamentos corretos

Em banco relacional, uma tabela normalmente **depende da outra**.

Exemplo clássico:

- Cliente faz pedidos
 - Pedido depende de um cliente
-

Criando tabela de pedidos com FK

```
CREATE TABLE pedido (  
    id_pedido SERIAL PRIMARY KEY,  
    data_pedido DATE NOT NULL,  
    valor_total NUMERIC(10,2) NOT NULL CHECK (valor_total >= 0),  
    id_cliente INTEGER NOT NULL,  
  
    CONSTRAINT fk_pedido_cliente  
        FOREIGN KEY (id_cliente)  
        REFERENCES cliente(id_cliente)  
);
```

📌 O que isso evita:

- ✅ Pedido com cliente inexistente
- ✅ Relatórios com “cliente desconhecido”
- ✅ Dados órfãos, que quebram dashboards

✓ 6. Comandos DML (movimentação de dados)

DML é o que o sistema faz o tempo todo:

- inserir
- consultar
- atualizar
- excluir

6.1 INSERT (inserir dados)

```
INSERT INTO cliente (nome, email, data_cadastro)
VALUES ('Ana Silva', 'ana@email.com', '2024-01-10');
```

📌 Situação real:
Cadastro de cliente vindo do site ou do caixa da loja.

6.2 SELECT (consultar dados)

```
SELECT nome, email
FROM cliente;
```

📌 Situação real:
Um analista pode usar SELECT para montar bases para:

- Power BI

- Excel
 - campanhas de marketing
-

6.3 UPDATE (atualizar dados)

```
UPDATE cliente  
SET email = 'ana.silva@email.com'  
WHERE id_cliente = 1;
```

 Situação real:
Correção de cadastro (comum em empresas).

⚠ O WHERE é obrigatório na prática, porque sem ele o UPDATE altera tudo.

6.4 DELETE (excluir dados)

```
DELETE FROM cliente  
WHERE id_cliente = 2;
```

 Situação real:
Exclusão pode acontecer por:

- duplicidade de cadastro
- regra de LGPD (em alguns casos)

⚠ delete também precisa de WHERE com cuidado.

7. WHERE: filtragem é a “alma” da análise de dados

O **WHERE** é o que transforma consulta em **informação útil**.

Sem WHERE, você retorna tudo.

Com WHERE, você responde perguntas reais.

7.1 Filtragem por comparação

```
SELECT *  
FROM pedido  
WHERE valor_total > 500;
```

 Pergunta do negócio:

➡ “Quais foram os pedidos acima de 500 reais?”

7.2 AND e OR

```
SELECT *  
FROM cliente  
WHERE data_cadastro >= '2024-01-01'  
AND email IS NOT NULL;
```

 Pergunta real:

➡ “Quais clientes cadastrados esse ano informaram e-mail?”

7.3 LIKE (padrões em texto)

Clientes que começam com A:

```
SELECT *  
FROM cliente  
WHERE nome LIKE 'A%';
```

 Pergunta real:

➡ “Quero testar uma amostra de clientes com nomes iniciando por A.”

7.4 IN (lista de valores)

```
SELECT *  
FROM pedido  
WHERE id_cliente IN (1, 4, 7);
```

 Pergunta real:

➡ “Quero ver compras de um grupo específico de clientes VIP.”

7.5 BETWEEN (intervalos)

```
SELECT *  
FROM pedido  
WHERE data_pedido BETWEEN '2024-01-01' AND '2024-01-31';
```

 Pergunta real:

➡ “Quais foram os pedidos em Janeiro?”

8. Outros recursos muito úteis no PostgreSQL

8.1 Lidando com datas (muito comum em relatórios)

Pedidos do mês atual:

```
SELECT *  
FROM pedido  
WHERE date_trunc('month', data_pedido) = date_trunc('month',  
CURRENT_DATE);
```

📌 Usado para:

- dashboards mensais
 - acompanhamento de metas
 - relatórios automáticos
-

8.2 Funções agregadoras (as mais usadas em análise)

Total de pedidos:

```
SELECT COUNT(*) AS total_pedidos  
FROM pedido;
```

Faturamento total:

```
SELECT SUM(valor_total) AS faturamento  
FROM pedido;
```

Ticket médio:

```
SELECT AVG(valor_total) AS ticket_medio  
FROM pedido;
```

Maior e menor compra:

```
SELECT MAX(valor_total) AS maior_compra,  
       MIN(valor_total) AS menor_compra  
FROM pedido;
```

📌 Isso aparece em quase todo dashboard:

- volume de vendas
- receita total

- ticket médio
 - melhores e piores resultados
-

8.3 CHECK: impedindo erros que “destroem” relatórios

Tabela de produto com regra de preço e estoque:

```
CREATE TABLE produto (  
    id_produto SERIAL PRIMARY KEY,  
    nome VARCHAR(120) NOT NULL,  
    preco NUMERIC(10,2) NOT NULL CHECK (preco > 0),  
    estoque INTEGER NOT NULL CHECK (estoque >= 0)  
);
```

📌 Impede que alguém salve:

- preço = 0
- estoque = -5

Sem isso, relatórios de estoque e faturamento ficam errados.

8.4 IS NULL e IS NOT NULL (limpeza e auditoria de dados)

Cientes sem email:

```
SELECT *  
FROM cliente  
WHERE email IS NULL;
```

📌 Pergunta real:

➡ “Quantos clientes estão incompletos e precisam de atualização?”

Fechamento: o que esse conhecimento permite no mundo real

Com isso, seu aluno já consegue:

- entender como tabelas são criadas e organizadas
- usar SQL para consultar e filtrar dados reais
- garantir consistência usando chaves e restrições
- preparar dados confiáveis para BI (Power BI, Excel, etc.)